

Folder

14 printable files

(file list disabled)

week10.py

```
1  #假设一段楼梯共15个台阶，小明一步最多能上3个台阶。编写程序计算小明上这段楼梯一共有多少种方法。
2
3  def stair(n):
4      #递归法
5      if n == 1:
6          return 1
7      elif n == 2:
8          return 2
9      elif n == 3:
10         return 4
11     else:
12         return stair(n-1) + stair(n-2) + stair(n-3)
13
14 print(stair(15))
15
16 def stair2(n):
17     #递推法
18     dp = [0] * (n + 1)
19     dp[1] = 1
20     dp[2] = 2
21     dp[3] = 4
22     for i in range(4, n + 1):
23         dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3]
24     return dp[n]
25
26 print(stair2(15))
```

week11.py

```
1  def hanoi(n, source_name, target_name, auxiliary_name, pegs):
2      if n == 1:
3          disk = pegs[source_name].pop()
4          pegs[target_name].append(disk)
5          print(f"移动盘子 {disk} 从 {source_name} 到 {target_name}")
6          print_pegs_status(pegs_initial_names, pegs)
7          return
8
9      hanoi(n - 1, source_name, auxiliary_name, target_name, pegs)
10
11     disk = pegs[source_name].pop()
12     pegs[target_name].append(disk)
13     print(f"移动盘子 {disk} 从 {source_name} 到 {target_name}")
14     print_pegs_status(pegs_initial_names, pegs)
15
16     hanoi(n - 1, auxiliary_name, target_name, source_name, pegs)
17
18 def print_pegs_status(peg_names_ordered, pegs_state):
19     for name in peg_names_ordered:
20         print(f" {name}: {pegs_state[name]}")
21     print("-" * 30)
22
23
24 if __name__ == "__main__":
25     num_disks = int(input("请输入盘子数量: "))
26     source_peg_name = 'A'
27     target_peg_name = 'C'
28     auxiliary_peg_name = 'B'
29     pegs = {
30         source_peg_name: list(range(num_disks, 0, -1)),
31         auxiliary_peg_name: [],
32         target_peg_name: []
33     }
34     pegs_initial_names = [source_peg_name, auxiliary_peg_name, target_peg_name]
35
36
37     print("初始状态:")
38     print_pegs_status(pegs_initial_names, pegs)
39
40     if num_disks > 0:
41         hanoi(num_disks, source_peg_name, target_peg_name, auxiliary_peg_name, pegs)
42     else:
43         print("没有盘子需要移动。")
44
45     print("最终状态:")
```

```

46     print_peg_status(pegs_initial_names, pegs)
47     total_moves = 2**num_disks - 1 if num_disks >=0 else 0
48     print(f"总共移动了 {total_moves} 步。")

```

week12.py

```

1  '''
2  请按要求自定义类并不断优化：
3  1. 设计一个学生Student类,类中属性包括id学号、name姓名、gender性别、age年龄；
4  具体设计要求如下：
5  (1) 设计Student的构造函数Student(id, name, gender, age) (通过学号、姓名、性别、年龄构造，其中id,name,gender为字符串，age为整型)
6  (2) 设计Student的方法getAge()返回该学生的年龄
7  (3) 设计Student的方法setAge(newage)更改该学生的年龄
8  (4) 定义Student的方法showInfo()显示学生的基本信息，返回格式化的字符串，具体格式如id:3, name:sam, gender:male, age:20
9  2. 请构造如下两个实例并显示信息
10 (1) '202201', '张三', 'male', 18
11 (2) '202202', '李四', 'female', 20
12 3. 修改张三的年龄为19岁，重新显示信息
13 4. 设置一个类变量，统计现有学生人数
14 5. 新增一个学生，'202203', '王五', 'female', 17
15 6. 打印现有学生人数
16 程序保存为ex3_student.py。
17 '''
18 class Student:
19     count = 0
20     def __init__(self, id, name, gender, age):
21         self.id = id
22         self.name = name
23         self.gender = gender
24         self.age = age
25         Student.count += 1
26
27     def getAge(self):
28         return self.age
29
30     def setAge(self, newage):
31         self.age = newage
32
33     def showInfo(self):
34         return f"id:{self.id}, name:{self.name}, gender:{self.gender}, age:{self.age}"
35
36 student1 = Student('202201', '张三', 'male', 18)
37 print(student1.showInfo())
38
39
40 student2 = Student('202202', '李四', 'female', 20)
41 print(student2.showInfo())
42
43
44 student1.setAge(19)
45 print(student1.showInfo())
46
47 student3 = Student('202203', '王五', 'female', 17)
48 print(student3.showInfo())
49
50 print("学生人数:", Student.count)

```

week13.py

```

1  '''
2  继承6.5节例6-2中的Person类生成Student类，编写新的函数用来设置学生专业，然后生成该类对象并显示信息。
3  '''
4
5  class Person(object):
6      def __init__(self, name):
7          assert isinstance(name, str), 'name must be string'
8          self.name = name
9
10     def sing(self):
11         print(self.name+' can sing.')
12     def walk(self):
13         print(self.name+' can walk.')
14     def eat(self):
15         print(self.name+' can eat.')
16
17 class Student(Person):
18     def __init__(self, name):
19         super().__init__(name)
20         self.major = None
21
22     def set_major(self, major):
23
24         assert isinstance(major, str), '专业名必须是字符串类型'

```

```

25         self.major = major
26
27     def show_info(self):
28
29         print(f"姓名: {self.name}")
30         print(f"专业: {self.major if self.major else '未设置专业'}")
31
32
33 if __name__ == '__main__':
34
35     student = Student("张三")
36
37
38     student.sing()
39     student.walk()
40
41     student.set_major("人工智能")
42     student.show_info()

```

week14.py

```

1  '''
2  编写一个程序demo.py，要求：
3  运行该程序后，生成demo_new.py文件，其中内容与demo.py一样，只是在每一行的后面加上行号。
4  要求行号以#开始，并且所有行的#垂直对齐。
5  '''
6  def func():
7
8      source_filename = __file__
9      target_filename = 'demo_new.py'
10
11      try:
12          with open(source_filename, 'r', encoding='utf-8') as f_source:
13              lines = f_source.readlines()
14
15              max_line_length = 0
16              for line in lines:
17
18                  if len(line.rstrip()) > max_line_length:
19                      max_line_length = len(line.rstrip())
20
21
22              new_lines = []
23              for i, line in enumerate(lines):
24                  line_content = line.rstrip()
25                  num_spaces = max_line_length - len(line_content) + 2
26
27
28                  new_line = f"{{line_content}}{' ' * num_spaces}# {{i + 1}}\n"
29                  new_lines.append(new_line)
30
31
32              with open(target_filename, 'w', encoding='utf-8') as f_target:
33                  f_target.writelines(new_lines)
34
35              print(f"成功生成文件: '{target_filename}'")
36
37          except FileNotFoundError:
38              print(f"错误: 源文件 '{source_filename}' 未找到。")
39          except Exception as e:
40              print(f"发生未知错误: {e}")
41
42 if __name__ == '__main__':
43     func()

```

week2.py

```

1  '''编写程序，用户输入一个三位以上的整数，输出其百位以上的数字。
2  例如用户输入1234，则程序输出12（提示：使用整除运算）。
3  '''
4
5  number = input("请输入一个三位以上的整数: ")
6  if len(number) < 3:
7      print("输入的数字不符合要求")
8      exit()
9  list1 = list(number)
10 list1.pop()
11 list1.pop()
12 outputnumber = "".join(list1)
13 print("输出为: ",outputnumber)
14
15

```

week3.py

```

1 | # 使用列表实现筛选法求素数
2 | # 编写程序，输入一个大于2的自然数，然后输出小于该数字的所有素数组成的列表。所谓素数，是指除了1和自身之外没有其他因数的自然数，最小的素数是2，后面依次是3 5 7
   | 11 13
3 | # 提示：算法考虑埃氏筛或者欧拉筛。
4 |
5 | n = int(input())
6 | primes = []
7 | candidates = list(range(2, n))
8 |
9 | while candidates:
10 |     p = candidates[0]
11 |     primes.append(p)
12 |     candidates = [num for num in candidates if num % p != 0]
13 |     if p * p > n:
14 |         primes += candidates
15 |         break
16 |
17 | print(primes)

```

week4.py

```

1 | #编写程序，输入一个字符串，输出其中出现次数最多的字符及其出现的次数。要求使用字典。
2 | # from collections import Counter
3 | # word = input("请输入一个字符串: ")
4 | # print(word)
5 | # frequencies = Counter(word)
6 | # frequencies.items()
7 | # print(frequencies.most_common(1))
8 |
9 | word = input("请输入一个字符串: ")
10 |
11 | word_dict = {}
12 | for i in word:
13 |     if i in word_dict:
14 |         word_dict[i] += 1
15 |     else:
16 |         word_dict[i] = 1
17 |
18 | max_char = ''
19 | max_count = 0
20 | for k, v in word_dict.items():
21 |     if v > max_count:
22 |         max_char = k
23 |         max_count = v
24 |
25 | print(max_char)

```

week5.py

```

1 | #使用集合实现筛选法求素数。输入一个大于2的自然数，输出小于该数字的所有素数组成的集合。
2 | #提示：请自学并使用集合的discard方法。
3 | import time
4 | n = int(input())
5 | primes = set()
6 | candidates = set(range(2, n))
7 | start = time.time()
8 |
9 | while candidates:
10 |     p = min(candidates)
11 |     primes.add(p)
12 |     i = p
13 |     while i < n:
14 |         candidates.discard(i)
15 |         i += p
16 |     if p * p > n:
17 |         primes.update(candidates)
18 |         break
19 |
20 | print(primes)
21 | print(time.time()-start)

```

week6.py

```

1 | x = int(input("x = "))
2 |
3 | if (x < 0):
4 |     print("y = ", 0)
5 | elif (0 <= x < 5):
6 |     print("y =", x)
7 | elif (5 <= x < 10):
8 |     print("y =", 3*x - 5)
9 | elif (10 <= x < 20):
10 |     print("y =", 0.5*x - 2)
11 | else:

```

```
12 |     print("y =", 0)
```

week7.py

```
1 | #1955年，卡普耶卡对4位数字进行了研究，发现一个规律：对任意各位数字不相同的4位数，使用各位数字能组成的最大数减去能组成的最小数，对得到的差重复这个操作，最终会得到6174这个数字，并且这个操作最多不会超过7次。
2 | #编写程序，使用枚举法对这个猜想进行验证。
3 |
4 | def kpyk(strnum):
5 |     minnum = int("".join(sorted(str(strnum))))
6 |     maxnum = int("".join(sorted(str(strnum), reverse=True)))
7 |     result = maxnum - minnum
8 |     return result
9 |
10 |
11 |
12 | # strnum = input("请输入一个任意个位数字都不相同的四位数:")
13 | # number = int(strnum)
14 | # if number > 10000 or number < 1000:
15 | #     print("您输入的不是四位数。")
16 | # if len(strnum) != len(set(strnum)):
17 | #     print("您输入的不是一个任意个位数字都不相同的四位数。")
18 |
19 | numbers = []
20 |
21 | for num in range(1000, 10000): # 所有四位数
22 |     strnum = str(num)
23 |     if len(set(strnum)) == 4: # 每位都不相同
24 |         numbers.append(num)
25 |
26 | # minnum = int("".join(sorted(strnum)))
27 | # maxnum = int("".join(sorted(strnum, reverse=True)))
28 |
29 | for number in numbers:
30 |     result = kpyk(number)
31 |     for i in range(1,8):
32 |         if result != 6174:
33 |             result = kpyk(result)
34 |         else:
35 |             break
36 |
37 |     if result != 6174:
38 |         print("error")
39 |     else:
40 |         print(number, " ", result)
41 |         print("true")
42 |
43 |
44 | # result = kpyk(strnum)
45 |
46 | # for i in range(1,7):
47 | #     if result != 6174:
48 | #         result = kpyk(result)
49 | #         i = i + 1
50 | #     else:
51 | #         break
52 |
53 | # if result != 6174:
54 | #     print("error")
55 | # else:
56 | #     print(result)
57 | #     print("true")
58 |
59 |
60 |
61 |
62 |
63 |
64 |
```

week8-1.py

```
1 | #一般地，可以作为密码字符的主要有数字、小写字母、大写字母和几个标点符号。
2 | #密码安全强度主要和字符串的复杂程度有关系，字符串中包含的字符种类越多，认为其安全强度越高。
3 | #按照这个标准，可以把安全强度分为强密码、中高、中低、弱密码。
4 | # 其中强密码表示字符串中同时含有数字、小写字母、大写字母、标点符号这4类字符，而弱密码表示字符串中仅包含4类字符中的一种。
5 | #编写程序，输入一个字符串，输出该字符串作为密码时的安全强度。
6 |
7 |
8 | import string
9 |
10 | passwords = input('Password:')
11 |
```



```
2 |
3 | def find_three_letter_words(text):
4 |
5 |     words = re.findall(r'\b\w{3}\b', text.lower())
6 |     return words
7 |
8 | if __name__ == "__main__":
9 |     english_text = input("请输入一段英文文本: ")
10 |    three_letter_words = find_three_letter_words(english_text)
11 |    if three_letter_words:
12 |        print("这段英文中长度为3的单词有: ")
13 |        for word in three_letter_words:
14 |            print(word)
15 |    else:
16 |        print("这段英文中没有长度为3的单词。")
```